

Debugging Engine Enhanced by Prior Knowledge: Can We Teach LLM How to Debug?

KUNYI LI, Zhejiang University, China

SAI WU, Zhejiang University, China

XIU TANG*, Zhejiang University, China

CHANG YAO, Zhejiang University, China

SONGHAO BU, Zhejiang University, China

QUANQING XU, Ant Group, China

GANG CHEN, Zhejiang University, China

Automated Program Repair (APR) powered by Large Language Models (LLMs) has shown strong potential for improving software reliability. However, existing LLM-based APR approaches underutilize the rich debugging knowledge latent in large-scale bug-fix corpora. Prior work has primarily advanced APR through multi-agent coordination, prompt engineering, task decomposition, or model training. While effective to some extent, these methods rely heavily on the implicit reasoning capacity of LLMs, without explicitly modeling the debugging knowledge that underlies successful program repair. We present DeepK, a novel framework that systematically extracts, validates, and reuses debugging knowledge to guide LLMs in APR. Rather than treating historical bug-fix pairs solely as contextual exemplars, DeepK distills them into a structured knowledge base of verified debugging knowledge. This knowledge base can be seamlessly integrated into diverse APR pipelines, providing interpretable reasoning traces and step-wise repair strategies that enhance the repair performance.

We evaluate DeepK on multiple benchmarks (ACPR, Atcoder) using both GPT-4o and DeepSeek-v3. Results show that DeepK consistently surpasses state-of-the-art APR systems in repair accuracy. Ablation studies confirm the importance of edit description generation, multi-perspective retrieval, and two-fold debugging knowledge. Varying the number of retrieved entries further highlights the trade-off between informativeness and noise. These findings emphasize the essential contribution of explicit debugging knowledge in advancing LLM-based APR and establish DeepK as an extensible framework for building more reliable repair systems.

CCS Concepts: • **Software and its engineering** → **Software testing and debugging**.

Additional Key Words and Phrases: Automated Program Repair, Retrieval-Augmented Generation, Large Language Models

ACM Reference Format:

Kunyi Li, Sai Wu, Xiu Tang, Chang Yao, Songhao Bu, Quanqing Xu, and Gang Chen. 2026. Debugging Engine Enhanced by Prior Knowledge: Can We Teach LLM How to Debug?. *Proc. ACM Softw. Eng.* 3, FSE, Article FSE082 (July 2026), 21 pages. <https://doi.org/10.1145/3797110>

*Corresponding author

Authors' Contact Information: [Kunyi Li](mailto:lkyzju@zju.edu.cn), Zhejiang University, Hangzhou, China, lkyzju@zju.edu.cn; [Sai Wu](mailto:wusai@zju.edu.cn), Zhejiang University, Hangzhou, China, wusai@zju.edu.cn; [Xiu Tang](mailto:tangxiu@zju.edu.cn), Zhejiang University, Hangzhou, China, tangxiu@zju.edu.cn; [Chang Yao](mailto:changy@zju.edu.cn), Zhejiang University, Hangzhou, China, changy@zju.edu.cn; [Songhao Bu](mailto:songhao_bu@zju.edu.cn), Zhejiang University, Hangzhou, China, songhao_bu@zju.edu.cn; [Quanqing Xu](mailto:xuquanqing.xqq@antgroup.com), Ant Group, Hangzhou, China, xuquanqing.xqq@antgroup.com; [Gang Chen](mailto:cg@zju.edu.cn), Zhejiang University, Hangzhou, China, cg@zju.edu.cn.



This work is licensed under a Creative Commons Attribution 4.0 International License.

© 2026 Copyright held by the owner/author(s).

ACM 2994-970X/2026/7-ARTFSE082

<https://doi.org/10.1145/3797110>

1 Introduction

Automated Program Repair (APR) aims to detect and fix bugs in source code with minimal human intervention [29]. Given that debugging and maintenance may consume up to 80% of software development effort [23], improving the efficiency and reliability of repair is of practical significance. Traditional APR techniques often rely on heuristic rules or constraint-based solvers, which face challenges in terms of scalability and generalization [18, 21, 22, 25–27, 29, 36]. With the advent of Large Language Models (LLMs), the field has witnessed remarkable progress, leveraging LLMs’ strong capabilities in parsing natural language, understanding developer intent, and generating high-quality code [8, 10, 49, 53, 56]. Early efforts such as TFix [3] in 2021 demonstrated the potential of framing program repair as a text-to-text translation task using models like T5 [33], while later systems such as CIRCLE [50] and AlphaRepair [43] explored cloze-style patch generation through fine-tuning or zero-shot prompting strategies. More recent research like AutoCodeRover [54], AgentLess [42], ThinkRepair [49] has gone beyond simple prompting, incorporating richer contextual information to better guide LLMs during the debugging process.

While these methods have achieved encouraging results, most still primarily rely on the LLM’s implicit pattern recognition capabilities to infer debugging strategies from raw examples or ad hoc prompts, rather than providing explicit and structured guidance on how to debug. This gap motivates our central question: can we explicitly teach LLMs how to debug by distilling and operationalizing debugging knowledge extracted from historical bug-fix instances? Retrieval-Augmented Generation (RAG) provides a lightweight mechanism for injecting external knowledge at inference time, mitigating hallucination and improving adaptability of LLMs [4, 11, 12, 14, 24, 34, 37, 51]. However, prior retrieval-based APR methods typically surface raw artifacts (e.g., similar code, documents) without transforming them into structured, semantically meaningful debugging guidance.

In this paper, we propose a novel RAG-based APR framework DeepK (**D**ebugging **E**ngine **E**nhanced by **P**rior **K**nowledge), which, unlike other LLM-based APR systems, explicitly equips LLMs with debugging knowledge extracted from existing code datasets through a four-phase pipeline. **(1) Edit Description Generation.** From <buggy, fixed> code pairs, we first generate natural language descriptions of code edits. This step translates raw code diffs into human-readable explanations, providing a bridge between low-level changes and high-level debugging reasoning. **(2) Structured Knowledge Extraction.** Guided by the generated edit descriptions, we extract structured debugging knowledge, decomposing each instance into two complementary components: (i) problem understanding, which captures the root cause of the bug, and (ii) debugging strategy, which outlines a concrete sequence of repair actions. **(3) Debugging Knowledge Base Construction.** The extracted knowledge is systematically organized into a knowledge base using multi-perspective embeddings (e.g., task description, abstract syntax tree (AST) structure, code trace). **(4) Prior Knowledge Enhanced Debugging.** During inference, the knowledge base is leveraged through a RAG mechanism to guide repair. Retrieved knowledge provides explicit debugging cues, helping LLMs better diagnose bugs and produce precise, actionable fixes.

We implement two variants of DeepK and evaluate them against five representative LLM-based APR systems: ThinkRepair [49], Self-Repair [31], LDB [56], AgentCoder [15], and AutoCodeRover [54]. Experimental results demonstrate that DeepK consistently improves repair accuracy across both in-distribution and out-of-distribution benchmarks. Ablation studies highlight the critical role of carefully designed components. Analysis of the number of retrieved knowledge entries further reveals a trade-off between informativeness and noise. Together, these findings position DeepK as a principled framework for augmenting LLMs with explicit, reusable debugging knowledge. We make the following contributions in this work:

- **A Novel LLM-based APR Framework:** We propose DeepK, a RAG-based APR framework that leverages a well-designed debugging knowledge base. Our method consistently outperforms state-of-the-art baselines across multiple experiment settings.
- **Debugging Knowledge Extraction and Utilization:** We introduce a systematic pipeline to extract, organize, and utilize debugging knowledge from existing code datasets. Our study demonstrates that incorporating such structured knowledge substantially enhances the reasoning and repair capabilities of LLMs.
- **Extensive Experimental Validation:** We design a comprehensive experimental suite spanning both in-distribution and out-of-distribution scenarios, providing deep insight into the effectiveness of debugging knowledge for LLM-based APR systems.

The structure of this paper is as follows: In Section 2, we provide a notation of key concepts utilized in this work, review related works and clarify the problem scope of this work. In Section 3, we present a detailed description of our framework, phase by phase, highlighting the main innovations. Section 4 outlines the datasets, baseline models, implementation and evaluation metrics employed in our experiments, followed by Section 5, where we address our research questions through a series of comprehensive experiments. Finally in Section 6 and 7, we discuss the limitations of our approach and conclude the paper.

2 Related Work and Background

2.1 Notation

To ensure clarity and eliminate ambiguity, we define the following terms used throughout the paper:

- **buggy code:** A single code file intended to implement a specific functionality but contains one or more bugs.
- **fixed code:** A corrected version of the buggy code that successfully implements the intended functionality. It is always associated with a corresponding buggy code.
- **<buggy, fixed> code pairs:** Pairs of buggy and fixed code snippets that accomplish the same task. These code snippets are largely similar in structure and content, differing mainly in the lines responsible for the bug and its fix.
- **debugging code dataset:** A dataset composed of multiple <buggy, fixed> code pairs, each accompanied by contextual metadata such as task descriptions, test cases, expected outputs, and other relevant information.

2.2 Related Works

Research on APR has explored diverse ways to leverage LLMs for bug localization and patch generation. We categorize prior work by the degree of autonomy granted to LLMs: **Agentic** approaches, where models actively plan and execute repair strategies, generally targeting repository-level bugs requiring multi-step reasoning, tool integration, and dynamic exploration, and **Workflow** approaches, which constrain LLMs to fixed, deterministic workflows, focusing on function-level or single-file repairs where deterministic processes suffice.

2.2.1 Agentic-Based Approaches. Agentic approaches treat LLMs as autonomous agents capable of dynamically planning repair steps, invoking external tools, and interacting with their environment to achieve a correct patch [5, 13, 15, 20, 38, 39, 54]. For example, RepairAgent [5] frames the LLM as an agent guided by a finite-state machine, equipped with code manipulation and testing tools to autonomously gather context, propose and validate patches, and iterate until the bug is resolved. AutoCodeRover [54] employs a structured retrieval strategy using AST search APIs

to iteratively collect relevant program elements across a repository, combined with spectrum-based fault localization when test suites are available. MAGIS [38] and similar agentic frameworks leverage tool integration and step-by-step reasoning to tackle repository-level bugs. Industrial systems such as the BRT Agent [9] further demonstrate this paradigm by autonomously generating bug-reproduction tests (BRTs) from bug reports and integrating them into repair workflows to enhance patch validation.

Agentic methods closely mimic human debugging practices and exhibit strong adaptability to complex repair scenarios. However, they incur higher computational and engineering overhead, are harder to reproduce, and may suffer from failure cascades due to unreliable tool usage or hallucinated reasoning. Despite these challenges, agentic approaches represent a promising direction for scaling APR to real-world, large-scale software systems.

2.2.2 Workflow Approaches. Workflow approaches, on the contrary, rely on *predefined workflows*, constraining LLMs to deterministic prompting, retrieval, or training pipelines without granting them full autonomy over task execution. These methods are typically simpler, more reproducible, and well-suited to localized bug-fixing scenarios.

Prompting and Retrieval-Augmented Methods. Prompt-based APR pipelines use structured prompts, few-shot examples, or chain-of-thought reasoning to guide patch generation. RAG techniques further enhance repair capability by retrieving semantically similar <buggy, fixed> code pairs, API documentation, or structured representations (e.g., control-flow and data-flow graphs) to provide debugging context [28, 49, 53]. While these methods effectively improve model reasoning with external knowledge, they rely on external orchestration and do not involve autonomous planning.

Scripted and Loop-Based Pipelines. Several approaches employ deterministic repair loops, such as alternating patch generation with regression test execution [19, 31, 44], incorporating human feedback [46, 55], or augmenting workflows with formal analysis and knowledge graphs [42, 47, 56]. These systems enhance repair reliability while maintaining predictable behavior.

Training-Based Approaches. Another line of research fine-tunes or trains LLMs explicitly for program repair, leveraging large corpora of bug-fix pairs. Such methods include full model fine-tuning, parameter-efficient tuning, and reinforcement learning [17, 35, 57], achieving high accuracy on benchmark datasets. But this demands substantial computational resources for fine-tuning or reinforcement learning, and lacks flexibility, since adapting to new coding practices requires retraining, making it hard to incorporate new external knowledge compared to retrieval-augmented approaches.

It is important to note that these strategies are not mutually exclusive; elements of different approaches can be combined within a single system. Workflow approaches remain attractive for their reproducibility and efficiency, particularly in single-function bug scenarios where sophisticated orchestration is unnecessary.

2.3 Motivation

Despite recent advances in leveraging LLMs for automated program repair APR, existing approaches have yet to fully harness the rich debugging knowledge embedded in large-scale code datasets. Most prior efforts focus on improving repair performance through multi-agent coordination, prompt engineering, task decomposition, or model training. While the training-based methods suffer from high cost and limited generalization, other strategies primarily rely on contextual cues or orchestration mechanisms by providing supplementary information and a clearer repair workflow, rather than explicitly teaching models the reasoning process required for debugging.

Existing RAG-based methods, which retrieve relevant code snippets, documentation, or structural information, surface raw context but do not extract or encode structured debugging knowledge. Consequently, LLMs must infer repair strategies implicitly from retrieved information. For example, ThinkRepair[49] constructs chain-of-thought (CoT)-style demonstrations from <buggy, fixed> code pairs to guide LLM reasoning via in-context learning. However, it relies on simple prompting templates rather than explicit modeling of debugging strategies. AuPairs[28] focuses on curating high-quality (“golden”) code pairs to improve example selection for repair, yet it offers no insight into *how* bugs are located or fixed, leaving the reasoning burden entirely on LLM pattern recognition.

To address these limitations, we propose DeepK, the first LLM-based APR framework that explicitly extracts and integrates structured debugging knowledge from large-scale datasets of <buggy, fixed> code pairs. Rather than simply retrieving similar examples or relevant context, DeepK builds a structured knowledge base of debugging knowledge, capturing both bug understanding and step-by-step debugging strategies, to enable knowledge-driven program repair.

2.4 Problem Scope

In this work, we deliberately focus on the **function-level** APR setting, where the bug is confined to a single function or a standalone file. This focus is motivated by our primary research objective: to investigate whether leveraging prior debugging knowledge can substantially enhance the reasoning and repair capabilities of LLMs. Addressing repository-level APR would necessarily involve extensive system engineering, multi-agent planning, and integration of numerous external tools, which risks conflating our investigation of LLM reasoning with challenges in large-scale system design and orchestration. By isolating the function-level setting, we ensure that improvements in repair performance can be more directly attributed to enhanced model debugging rather than to complex pipeline engineering.

3 Approach

DeepK is a four-phase RAG-based APR framework (as shown in Figure 1) that (1) converts <buggy, fixed> code pairs into edit descriptions, (2) extracts structured debugging knowledge separating problem understanding and repair strategy, (3) organizes the knowledge into a multi-perspective knowledge base, and (4) retrieves and injects relevant knowledge at inference time to explicitly guide LLM-based program repair. In the following section we go into details about the phases of DeepK.

3.1 Phase 1: Edit Description Generation

The first two phases of our framework focus on extracting debugging knowledge from existing <buggy, fixed> code pairs, which serves as the foundation for subsequent retrieval and reuse. A key challenge, however, is ensuring that the extracted knowledge accurately reflects the true bug. Prior studies have shown that LLMs reason more effectively when guided through step-wise processes, rather than directly interpreting low-level syntactic representations such as raw code [16, 30, 40]. Motivated by this, we introduce **EditDescriptor**, an AST-based code differencing module that converts holistic comparisons between buggy and fixed code into natural-language, step-wise edits, thereby guiding the LLM toward identifying the root cause of the fault.

Given a <buggy, fixed> code pair, EditDescriptor parses both versions into ASTs, computes their differences using a tree edit distance (TED) algorithm, and translates the resulting edit actions into structured natural-language templates (Fig. 2, Algorithm 1). We extend the classical Tree-Edit-Distance algorithm [52] in two aspects. First, we exploit the AST structure to preserve the relative position of nodes, enabling precise localization of edits. Second, we filter out semantically unreasonable edit actions produced by TED. This is necessary because TED optimizes for structural

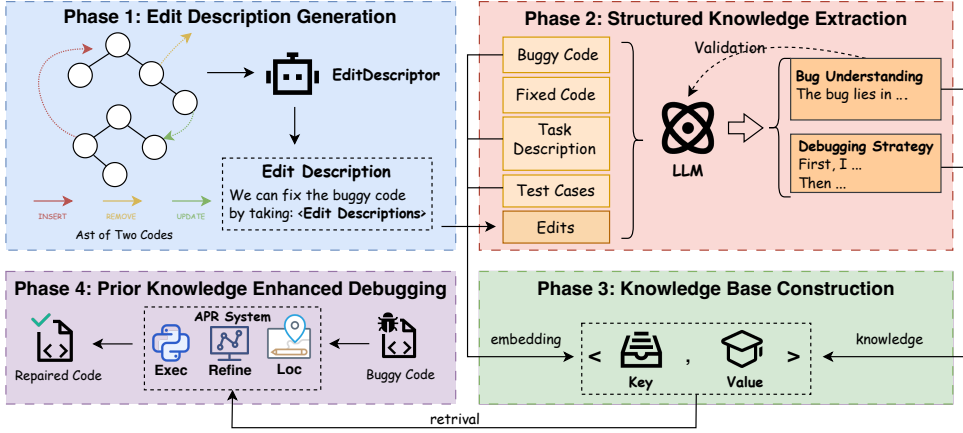


Fig. 1. The Four-Phase Framework of DeepK. The first two phases extract validated debugging knowledge from debugging code datasets, forming the knowledge base that is constructed in Phase 3. Then in Phase 4, this knowledge is leveraged to enhance the LLM-based APR system.

Algorithm 1 EditDescriptor

Require: buggy code: C_b , fixed code: C_f

Ensure: edit description: E_{nl}

$ast_b = astParse(C_b)$

$ast_f = astParse(C_f)$

$E_{structured} = AstEditDistance(ast_b, ast_f)$

$E_{nl} = TransformIntoNL(A_{structured})$

return E_{nl}

similarity rather than semantic consistency: code elements that are syntactically or structurally close in the AST may be semantically far apart, leading to implausible edits that are difficult for humans to interpret. To address this issue, we incorporate semantic information of code elements into node distance computation and forbid update operations between semantically incompatible nodes.

3.2 Phase 2: Structured Knowledge Extraction

DeepK extracts implicit debugging knowledge from <buggy, fixed> code pairs by instructing the LLM to answer the question: “Given the buggy code, how would you identify and fix the bug?”.

To ensure high-quality and context-aware reasoning, DeepK leverages the edit descriptions, enriched with additional contextual signals (e.g., test outcomes, execution traces, and associated metadata) from the debugging code datasets. These inputs are incorporated into carefully designed prompts to guide the LLM in producing structured debugging knowledge, which captures both bug understanding and step-by-step debugging strategies (see Fig. 3).

Following reasoning generation, DeepK performs a verification stage to rigorously assess the validity of the extracted knowledge. Specifically, the LLM is tasked with repairing the buggy code solely using its previously articulated debugging knowledge. The correctness of the resulting patch is then evaluated against ground-truth tests or reference fixes. Only debugging knowledge that

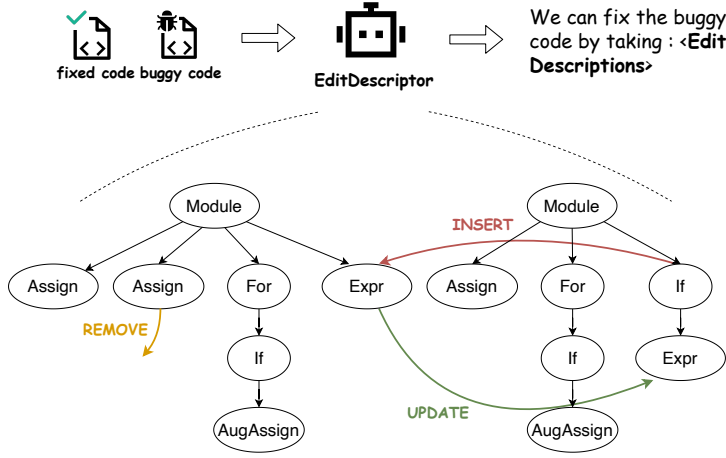


Fig. 2. Workflow of EditDescriptor. EditDescriptor takes <buggy, fixed> code pairs as input, parses them into AST structures and generates the edits needed to fix the buggy code in natural language.

Algorithm 2 Knowledge Extraction

Require: buggy code: C_b , edit description: E_{nl} , relevant metadata: M_C

Ensure: debugging knowledge

$prompt = PromptBuilder(C_b, E_{nl}, M_C)$

$BugUnderstanding, DebuggingStrategy = LLM(prompt)$

$C_{validate} = LLM(C_b, BugUnderstanding, DebuggingStrategy)$

$ValidOrNot = Exec(C_{validate}, M_C.test_case)$

if $Valid$ **then**

return $BugUnderstanding, DebuggingStrategy$

else

return $None, None$

end if

consistently yield correct repairs are admitted into the knowledge base, ensuring both precision and reliability of the extracted knowledge (detailed in Algorithm 2).

3.3 Phase 3: Debugging Knowledge Base Construction

To ensure effective retrieval of relevant debugging knowledge, we embed each knowledge instance into a three-fold index and store it in our knowledge base. The knowledge base can be noted as:

$$KB = \{ \langle I, K \rangle \mid I = (E_{task}, E_{AST}, E_{code}), K = (K_{understanding}, K_{strategy}) \}$$

Specifically, the index consists of:

- **Task description:** the original programming problem or functionality specification associated with the code. This reflects task-level similarity, ensuring that retrieved knowledge comes from code written to solve similar problems.

	Code To Fix	Here I have a buggy code snippet that I need to fix and the fixed code: ### <buggy, fixed> code pair {<buggy, fixed> code pair }
	Relevant Info	### Other Info {task_description / input / output / actual_output}
	Human-like Edits	The code can be fixed by taking the following actions: ### Actions {actions}
	Instructions	Now I need you tell me if you are given the buggy code snippet, the input, the expected output and the actual output, how would you analyze the code to find the bug? Note that the bug can be fixed by taking the actions listed above, so you should first understand what the bug is.

Bug Understanding	The bug is a logical error in the ...	
Debugging Strategy	1. First, I read the problem statement carefully to understand the requirements ... 2. I then looked at the buggy code ... 3. The actual output for input ... is ... because However, the expected output is ... 4. The discrepancy between the actual output and the expected output confirms that ... 5. The fix involves ...	

Fig. 3. Instruct LLM to Generate Debugging Knowledge.

- **AST structure:** a representation of the abstract syntax tree, capturing structural code similarity. This ensures that even when variable names or stylistic differences vary, knowledge from syntactically similar buggy contexts can still be retrieved.
- **Executed code lines:** runtime execution traces that highlight the lines most correlated with the bug. This dimension helps narrow down retrieval to knowledge relevant to the likely fault locations.

This multi-dimensional indexing strategy enables retrieval to be guided not only by surface-level textual similarity but also by structural and semantic properties of the bug.

Each knowledge entry contains two parts:

- **Bug Understanding:** A high-level reasoning trace and an explicit characterization of the bug's likely root cause.
- **Debugging Strategy:** A step-by-step debugging strategy, detailing the thought process.

For a given query (i.e., a new buggy instance), we retrieve candidate knowledge items independently along each dimension and then integrate these results by adopting Reciprocal Rank Fusion (RRF) [45], a rank aggregation method that combines rankings from multiple retrieval sources. Given a candidate knowledge item d , let $r_i(d)$ denote its rank from the i -th index. RRF assigns a fused score as:

$$score(d) = \sum_{i=1}^m \frac{1}{k + r_i(d)},$$

where m is the number of ranking sources (here, $m = 3$), and k is a small constant (commonly set to 60) to smooth rank differences. Items highly ranked across multiple indices receive higher fused scores, ensuring that retrieval is balanced across textual, structural, semantic, and categorical cues.

Table 1. The Size of Datasets.

Dataset	# Test	# Train
ACPR	196	793
ACPR-Hard	200	800
Atcoder-ABC	100	\
Atcoder-ARC	177	\
Atcoder-AGC	181	\

3.4 Phase 4: Prior Knowledge Enhanced Debugging

In the final phase, we leverage the constructed debugging knowledge base to guide the repair of new buggy code. When a buggy code snippet is presented, our system retrieves the most relevant knowledge entries based on the multi-dimensional embedding.

The structured debugging knowledge can be integrated into APR systems that incorporate an explicit reasoning module, namely a step that takes buggy code as input and produces natural-language hypotheses, diagnoses, or repair plans as output, which is a common module for a broad range of LLM-based APR systems [5, 19, 28, 31, 44, 48, 49, 56]. Systems that lack such a reasoning module are not directly compatible; however, the knowledge base can be reformatted into well-structured exemplars suitable for any framework that exposes a “buggy code $\rightarrow$ reasoning” interface, thereby enabling principled extension as new systems are developed.

By conditioning the LLM on this curated and validated debugging knowledge, the repair process shifts from single-instance reasoning (inferring fixes solely from the input bug) to a knowledge-guided reasoning paradigm, in which the model leverages prior knowledge distilled from a large, verified corpus of bug-fix pairs.

4 Experiment Settings

4.1 Datasets

There are two datasets used in our experiment: ACPR for in-distribution setting and Atcoder for out-of-distribution setting.

ACPR. ACPR[10] is a dataset collected from CodeNet [32]. We split it into two parts: a normal subset consisting of randomly sampled programs, and a hard subset consisting of the longest programs. We adopt a 4:1 train-test split and enforce strict partitioning to prevent data leakage: solutions to the same coding problem are placed exclusively in either the training or test set.

Atcoder. Atcoder is constructed from programming problems on AtCoder [1], covering contests of varying difficulty (ABC, ARC, AGC). To control temporal leakage into LLMs, 61.5% of the collected problems were released after Oct. 2023 (unseen by GPT-4o), and 41.7% after Jul. 2024 (unseen by GPT-4o and DeepSeek-v3). These problems are excluded from training, ensuring a strictly unseen evaluation scenario.

The details of all datasets are summarized in Table 1. We exclude widely adopted datasets such as HumanEval[6] and MBPP[2], as their early release dates increase the likelihood of inclusion in LLM pretraining corpora. Furthermore, these datasets are comparatively simple and have been extensively investigated in prior research, making them less suitable for evaluating advanced APR capabilities.

4.2 Baseline

We compare DeepK against four workflow methods (ThinkRepair [49], Self-Repair [31], LDB [56] and AgentCoder [15]) and one agentic method (AutoCodeRover [54]), covering a spectrum of LLM-based APR paradigms:

- **ThinkRepair** constructs a knowledge pool of diverse reasoning traces collected with chain-of-thought (CoT) prompts, then uses selected examples to guide iterative bug fixing through test feedback.
- **Self-Repair** is a scripted, loop-based pipeline that enables LLMs to iteratively debug and repair their own outputs via execution, reasoning from failed tests, and patch refinement.
- **LDB** introduces a debugging framework that leverages runtime execution traces to guide code refinement. It decomposes programs into basic blocks, tracks runtime variable states, and prompts LLMs to assess each block's correctness with respect to the task description.
- **AgentCoder** formulates APR as a multi-agent process, with dedicated agents coordinating retrieval, execution, and verification to incrementally refine patches.
- **AutoCodeRover**, an agentic approach, employs structured context retrieval by combining AST-based code search and spectrum-based fault localization to assemble relevant context for LLM-based repair.

To ensure a fair evaluation centered on the contribution of *debugging knowledge*, we exclude training-based workflow methods, as they rely on fine-tuned or retrained models, whereas all other baselines operate on publicly available, off-the-shelf LLMs. Moreover, we disable feedback loops and iterative refinement modules in all baselines, establishing a controlled setting where performance differences cannot be attributed to variations in interaction depth but instead reflect the impact of debugging knowledge.

4.3 Implementation

We select ThinkRepair as the representative of prompting- and retrieval-based systems, and Self-Repair as the representative of scripted, loop-based systems, to implement two variants of DeepK.

The first version, **DeepK-T**, integrates our debugging knowledge base into ThinkRepair [49]. In this extension, we replace ThinkRepair's original reasoning traces with structured debugging knowledge, providing richer and more accurate reasoning guidance without altering its workflow. The second version, **DeepK-S**, builds upon Self-Repair [31]. We integrate DeepK during the reasoning phase of Self-Repair, where test failures are analyzed to guide subsequent repair attempts. Specifically, we leverage our debugging knowledge to help the LLM produce more accurate and structured explanations of failure cases, which are then used to inform patch refinement.

In both implementations, the core workflows of ThinkRepair and Self-Repair remain unchanged; our contribution lies in seamlessly augmenting them with structured debugging knowledge. The default number of knowledge entries retrieved is 3 for both DeepK-T and DeepK-S.

4.4 Evaluation Metrics

We adopt repair accuracy as the primary evaluation metric:

$$Accuracy = \frac{\#Correct\ Patch}{size(Dataset)}.$$

A patch is deemed correct if it passes all test cases associated with the dataset.

Table 2. Accuracy of DeepK-T and DeepK-S compared with ThinkRepair, Self-Repair, LDB, AgentCoder, and AutoCodeRover using GPT-4o and DeepSeek-v3.

		In-Distribution		Out-of-Distribution		
		ACPR	ACPR-Hard	Atcoder-ABC	Atcoder-ARC	Atcoder-AGC
GPT-4o	ThinkRepair	0.7466	0.5767	0.1700	0.0414	0.0184
	Self-Repair	0.7041	0.5600	0.2400	0.0678	0.0331
	LDB	0.6786	0.5200	0.1750	0.0960	0.0773
	AgentCoder	0.1607	0.1250	0.0000	0.0000	0.0000
	AutoCodeRover	0.4786	0.2676	0.1400	0.0408	0.0130
	DeepK-T	0.7653	0.5817	0.2233	0.0603	0.0166
	DeepK-S	0.7449	0.5800	0.2900	0.0989	0.0580
DeepSeek-v3	ThinkRepair	0.8418	0.6933	0.3667	0.1751	0.1179
	Self-Repair	0.8367	0.7133	0.4133	0.1601	0.0866
	LDB	0.8027	0.7250	0.2733	0.1525	0.1731
	AgentCoder	0.7466	0.6000	0.4067	0.2994	0.2486
	AutoCodeRover	0.8367	0.6450	0.3600	0.1412	0.1271
	DeepK-T	0.9056	0.7675	0.4200	0.2655	0.2155
	DeepK-S	0.8597	0.7375	0.4250	0.1864	0.1160

5 Experiment Results

In this section, we go into the details of our experiments and answer the following research questions:

- **RQ1:** How does integrating our knowledge base enhance existing APR systems?
- **RQ2:** What is the contribution of components within DeepK?
- **RQ3:** What is the effect of the number of retrieved knowledge on DeepK's performance?

5.1 RQ1: How does integrating our knowledge base enhance existing APR systems?

We evaluate all systems on both in-distribution datasets (ACPR and ACPR-Hard) and out-of-distribution datasets (Atcoder-ABC, Atcoder-ARC, Atcoder-AGC), using GPT-4o and DeepSeek-v3.

5.1.1 Comparison with Baselines. The detailed results are presented in Table 2, covering both in-distribution and out-of-distribution settings. Overall, the findings highlight not only the robustness of DeepK but also the varying behaviors of different baselines when paired with different LLM backbones.

In-Distribution Performance. On the in-distribution datasets, both variants of DeepK consistently outperform the baselines. With GPT-4o, DeepK-T achieves the highest accuracy on ACPR (76.5%), surpassing ThinkRepair (74.7%) and Self-Repair (70.4%). On the more challenging ACPR-Hard, DeepK-T attains 58.2%, slightly surpassing ThinkRepair (57.7%), the strongest baseline. With DeepSeek-v3, the improvements are more substantial: DeepK-T achieves 90.6% on ACPR, exceeding ThinkRepair (84.2%) by more than 6 percentage points, and reaches 76.8% on ACPR-Hard, outperforming LDB (72.5%) by over 4 points.

Out-of-Distribution Performance. The AtCoder datasets pose a significantly harder, out-of-distribution challenge, as they involve competitive programming tasks with unfamiliar structures and more complex algorithms. Across all methods, accuracy drops sharply relative to in-distribution

performance. Nevertheless, the results still prove the effectiveness of the debugging knowledge. With GPT-4o, DeepK-S achieves the best accuracy on Atcoder-ABC (29.0%), representing a 5-point improvement over Self-Repair (24.0%). For Atcoder-ARC and Atcoder-AGC, all systems perform poorly, and the margins between methods remain small. With DeepSeek-v3, DeepK-S delivers the strongest result on Atcoder-ABC (42.5%), clearly outperforming all baselines. For Atcoder-ARC and Atcoder-AGC, AgentCoder achieves the best performance; however, as discussed below, this behavior stems from its ‘special design’ rather than a general repair capability. Excluding this case, DeepK-T consistently surpasses all other baselines.

Discussion. These results suggest that structured debugging knowledge contributes to better APR performance across datasets of different difficulty levels. With GPT-4o, the integration of knowledge brings moderate improvements on in-distribution benchmarks and on Atcoder-ABC. However, it does not substantially improve outcomes on the harder out-of-distribution datasets Atcoder-ARC and Atcoder-AGC, where all methods perform at very low levels. In contrast, with the stronger DeepSeek-v3 backbone, the same knowledge integration yields larger gains across both in-distribution and out-of-distribution settings. This pattern indicates that the benefits of structured debugging knowledge are shaped not only by dataset characteristics but also by the reasoning capacity of the underlying LLM.

Two systems warrant special discussion. First, AgentCoder exhibits highly inconsistent behavior across backbones. With GPT-4o, it performs extremely poorly, yielding near-zero accuracy across all datasets. In contrast, with DeepSeek-v3, it achieves the best results on Atcoder-ARC and Atcoder-AGC. Upon closer inspection, we find that AgentCoder does not perform true program repair: its ‘fix’ operation corresponds to full code regeneration rather than targeted modification, meaning the LLM cannot access the buggy code during repair. Consequently, with GPT-4o, the regenerated code often fails to satisfy input-output requirements, leading to abnormally low accuracy. Conversely, with DeepSeek-v3, regeneration can outperform repair in cases where the buggy code is severely flawed, explaining its relatively stronger results in certain out-of-distribution settings.

The second case is AutoCodeRover, the only fully agentic framework evaluated. Its performance is generally unsatisfactory, particularly under GPT-4o. We attribute this to two factors: (1) the toolkit design of AutoCodeRover is tailored for complex repository-level tasks (e.g., multi-file reasoning and dependency handling), which are not aligned with the scope of single-function repair tasks considered here; and (2) agentic frameworks depend heavily on the underlying LLM’s capability to handle task routing and tool invocation, making them especially vulnerable to weaker backbones.

5.1.2 Relevant Improvement. To gain better insight into the effectiveness of the debugging knowledge, we compared two DeepK variants with the original systems in Figure 4 and Table 3. We report absolute improvements in accuracy (percentage points, pp) and relative improvements (%).

Overall Effectiveness. Across all settings, the introduction of debugging knowledge resulted in measurable gains. For the ThinkRepair→DeepK-T transition, improvements were most pronounced with DeepSeek-V3, yielding an average absolute increase of +7.6 pp (± 3.1 pp) and a relative improvement of +33.4% across datasets. In contrast, GPT-4o exhibited a more modest average gain of +1.9 pp (± 2.0 pp) and +14.1% relative. For the Self-Repair→DeepK-S transition, GPT-4o achieved larger gains (+3.3 pp, ± 1.0 ; +30.3%) compared to DeepSeek-V3 (+2.3 pp, ± 0.9 ; +11.1%). These results demonstrate that the effectiveness of debugging knowledge is contingent on both the baseline system and the underlying LLM backend.

In-distribution vs. Out-of-distribution Performance. Performance improvements were observed in both in-distribution and out-of-distribution settings, though the relative magnitude differed. Under DeepSeek-V3, ThinkRepair→DeepK-T yielded comparable absolute improvements

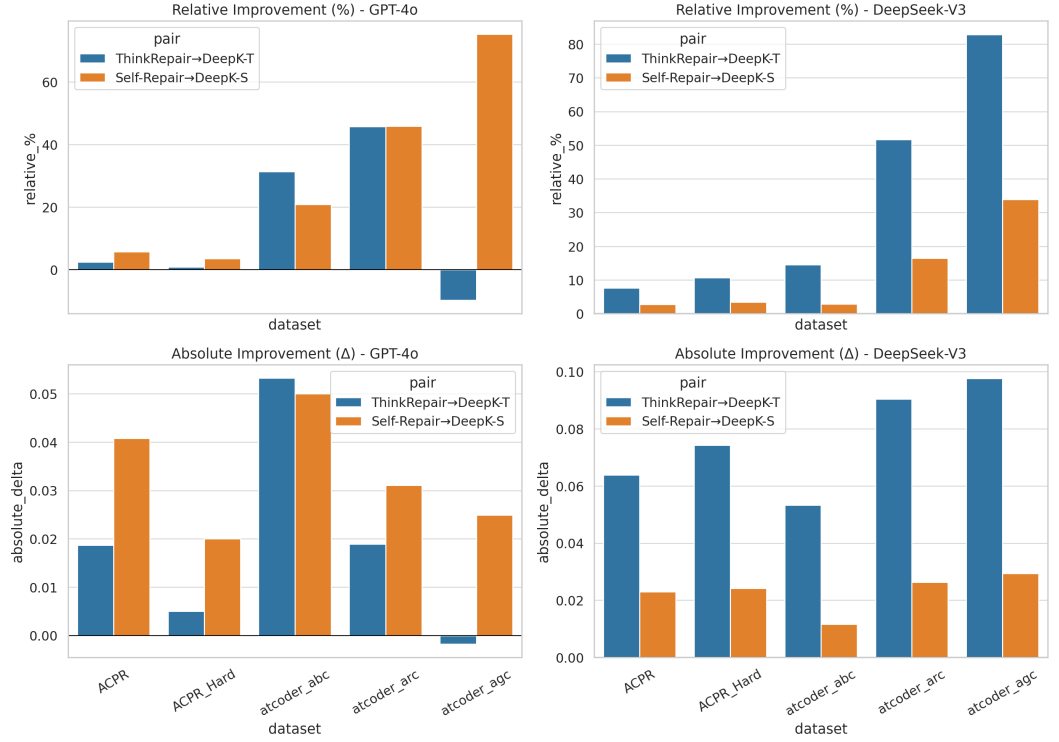


Fig. 4. The relative and absolute improvement of DeepK-T and DeepK-S over ThinkRepair and Self-Repair.

Table 3. The mean improvement across five datasets, with its standard deviation (std) showing variability.

Transition	Mean improvement (pp)	Std across datasets (pp)
ThinkRepair → DeepK-T (DeepSeek-V3)	+7.6	3.1
ThinkRepair → DeepK-T (GPT-4o)	+1.9	2.0
Self-Repair → DeepK-S (DeepSeek-V3)	+2.3	0.9
Self-Repair → DeepK-S (GPT-4o)	+3.3	1.0

in-distribution (+6.9 pp) and out-of-distribution (+8.0 pp). However, relative improvements were amplified in out-of-distribution evaluation (+49.6% vs. +9.1%) due to the lower baseline accuracy. A similar pattern was observed for GPT-4o Self-Repair→DeepK-S, where out-of-distribution datasets exhibited relative gains exceeding +47%, compared with only +4.7% in-distribution. These results suggest that debugging knowledge is particularly beneficial under distributional shift, where baseline systems struggle.

Statistical Significance. Despite overall positive trends, improvements were not uniform across datasets. For example, GPT-4o ThinkRepair→DeepK-T yielded a slight decrease on Atcoder-AGC (-0.18 pp, -9.8%). To assess robustness, we performed a Wilcoxon signed-rank test [41] over the five datasets for each system transition.

The Wilcoxon signed-rank test is a non-parametric method for comparing paired results. In our study, it was applied to evaluate whether the performance improvements from DeepK over

baseline systems are statistically consistent. A significant result indicates that DeepK systematically outperforms the baseline across datasets, rather than improvements being due to random variation.

Table 4. Wilcoxon signed-rank test across all 20 improvements (five datasets $\times$ four transitions). One-sided p -values test the hypothesis that DeepK outperforms the baseline.

W	$p_{\text{one-sided}}$	$p_{\text{two-sided}}$
1.0	1.9×10^{-6}	3.8×10^{-6}

Across 20 paired observations (5 datasets $\times$ 4 transitions), the test indicated a significant positive median improvement (one-sided $p < 0.001$ as shown in Table 4), supporting the conclusion that debugging knowledge systematically enhances APR performance.

5.2 RQ2: What is the contribution of components within DeepK?

5.2.1 Ablation Study on Multi-Dimensional Embeddings. We conducted an ablation study to quantify the contribution of each component of the three-fold retrieval index: task description, AST structure, and executed code lines. For this analysis, we used DeepK-T under the DeepSeek-v3backend as the representative configuration. The result is shown in Table 5.

Code trace embeddings are most critical on average. Removing executed-code-line embeddings produces the largest mean degradation across datasets:

$$\Delta_{\text{no-trace}} = \frac{1}{5} \sum_d (\text{score}_{\text{no-trace},d} - \text{score}_{\text{three-fold},d}) = -0.0192 \text{ (pp)}.$$

Per dataset, the drops are: ACPR−4.85 pp, ACPR-Hard−2.50 pp, Atcoder-ARC −1.97 pp, Atcoder-ARC −2.77 pp; an exception is Atcoder-ABC, where accuracy increases by +2.50 pp (0.4450 vs. 0.4200). These results indicate that runtime localization signals are highly informative for most settings, but may occasionally over-constrain retrieval on some out-of-distribution tasks.

Task description embeddings provide stabilizing task-level alignment. Removing task descriptions produces the smallest mean drop:

$$\Delta_{\text{no-desc}} = \frac{1}{5} \sum_d (\text{score}_{\text{no-desc},d} - \text{score}_{\text{three-fold},d}) = -0.0058 \text{ (pp)}.$$

The per-dataset changes are: ACPR−1.02 pp, ACPR-Hard−0.00 pp, Atcoder-ABC −0.50 pp, Atcoder-ARC −0.28 pp, Atcoder-ARC −1.11 pp. These results suggest the description index is a stabilizing signal that helps maintain task relevance, though it is not typically the dominant contributor when strong structural or execution cues are present.

Summary. The heterogeneous, dataset-dependent effects justify our use of RRF: by aggregating ranks from all three indices, the full system mitigates the weaknesses of any single dimension. Indeed, the three-fold model either attains or matches the per-dataset best in four of five settings and achieves the highest macro-level robustness.

However, both “no AST” and “no trace” exceed the full model on Atcoder-ABC (+2.5 pp), suggesting that additional structural/trace cues can sometimes induce over-specific retrieval that harms generalization. This highlights a potential opportunity for a better retrieval strategy to preserve the benefits of richer indices without sacrificing out-of-distribution robustness.

Table 5. Ablation study of the three-fold embedding for retrieval in DeepK-T (DeepSeek-V3 backend). Each row removes one embedding dimension while keeping the others. Bold numbers highlight the best score per dataset.

Method	ACPR	ACPR-Hard	Atcoder-ABC	Atcoder-ARC	Atcoder-AGC
Three-fold Embedding	0.9056	0.7675	0.4200	0.2655	0.2155
– no AST Embedding	0.8852	0.7525	0.4450	0.2203	0.1989
– no Description Embedding	0.8954	0.7675	0.4150	0.2627	0.2044
– no Code Trace Embedding	0.8571	0.7425	0.4450	0.2458	0.1878

Table 6. Ablation of knowledge components in DeepK-T (DeepSeek-v3). “Two-fold” uses both components. Δ denotes the *drop* in percentage points (pp) relative to Two-fold.

	ACPR	ACPR-Hard	Atcoder-ABC	Atcoder-ARC	Atcoder-AGC
Two-fold Knowledge	0.9056	0.7675	0.4200	0.2655	0.2155
Without Bug Understanding	0.8954	0.7350	0.4050	0.2571	0.1878
Δ (pp)	1.02	3.25	1.50	0.84	2.77
Without Debugging Strategy	0.8852	0.7425	0.3900	0.2599	0.1630
Δ (pp)	2.04	2.50	3.00	0.56	5.25

Table 7. Summary of absolute and relative drops (mean $\pm$ std) when removing each knowledge component, computed across five datasets.

Ablation	Mean drop (pp) $\pm$ std	Mean % drop $\pm$ std
Without Bug Understanding	1.88 $\pm$ 1.08	4.99% $\pm$ 4.55%
Without Debugging Strategy	2.67 $\pm$ 1.71	7.82% $\pm$ 9.47%

Table 8. Wilcoxon signed-rank tests (paired across 5 datasets) comparing Two-fold against ablated variants. One-sided tests use the alternative Two-fold > Ablated.

Comparison	W	$p_{\text{one-sided}}$	$p_{\text{two-sided}}$
Two-fold vs. Without Bug Understanding	15.0	0.03125	0.0625
Two-fold vs. Without Debugging Strategy	15.0	0.03125	0.0625

5.2.2 Ablation of Knowledge Components. We ablated the two structured knowledge components, *Bug Understanding* (root-cause characterization and high-level reasoning) and *Debugging Strategy* (step-by-step repair plan), to quantify their contributions to DeepK-T under DeepSeek-v3. As shown in Table 6, removing either component consistently reduces performance across all five datasets.

Removing *Bug Understanding* produces an average reduction of 1.88 pp (± 1.08), corresponding to a mean relative drop of 4.99% ($\pm 4.55\%$). The largest observed degradations occur on *ACPR-Hard*

Table 9. Ablation of the EditDescriptor. The values in the table denote the number of validated knowledge entries generated by Phase 2.

Setting	ACPR	ACPR-Hard
With edit description	774	736
Without edit description	768	718
Δ (#)	-6	-18

Table 10. Average rank of different numbers of retrieved knowledge entries across five datasets. Lower rank indicates better performance (rank 1 = best). Both DeepK-T and DeepK-S achieve peak performance with three entries.

#Entries	DeepK-T (avg. rank)	DeepK-S (avg. rank)
1	3.0	3.4
2	3.8	2.8
3	2.0	1.3
4	3.8	3.7
5	2.4	3.8

(-3.25 pp) and *Atcoder-AGC* (-2.77 pp). Removing *Debugging Strategy* produces a larger average reduction of 2.67 pp (± 1.71), or 7.82% ($\pm 9.47\%$) relative. The largest observed degradations occur on *Atcoder-AGC* (-5.25 pp) and *Atcoder-ABC* (-3.00 pp).

Paired Wilcoxon signed-rank tests across the five datasets (Table 8) show that Two-fold significantly outperforms each ablated variant under a one-sided alternative (Two-fold > Ablated; $p = 0.03125$ for both comparisons). Two-sided p -values are 0.0625 due to the discreteness of the exact test at $n = 5$. The rank-biserial correlation equals 1.0 in both cases (all paired differences positive), underscoring the uniformity of the effect.

Summary. Both components make statistically supported, complementary contributions: *Bug Understanding* stabilizes the reasoning space by anchoring root-cause hypotheses, while *Debugging Strategy* delivers larger gains by operationalizing those hypotheses into concrete repair steps. Conditioning the LLM on both yields the most reliable improvements, aligning with the overall design objective of shifting from single-instance reasoning to a knowledge-guided paradigm grounded in a curated corpus of validated debugging knowledge.

5.2.3 Ablation of Edit Descriptor. We ablated the EditDescriptor module to assess its contribution to knowledge extraction. As shown in Table 9, removing edit descriptions consistently reduced performance on both ACPR (774 $\rightarrow$ 768, -6) and ACPR-Hard (736 $\rightarrow$ 718, -18). The larger degradation on ACPR-Hard indicates that natural-language, step-wise edit descriptions are particularly valuable in more challenging settings, where they help guide the LLM toward the actual root cause of the bug rather than relying solely on raw code differencing.

5.3 RQ3: What is the effect of the number of retrieved knowledge on DeepK's performance?

We further examined the effect of varying the number of retrieved knowledge entries on system performance. Figure 5 presents the results of DeepK-T and DeepK-S across different knowledge entry number under DeepSeek-v3, while Table 10 summarizes the average ranks of each configuration.

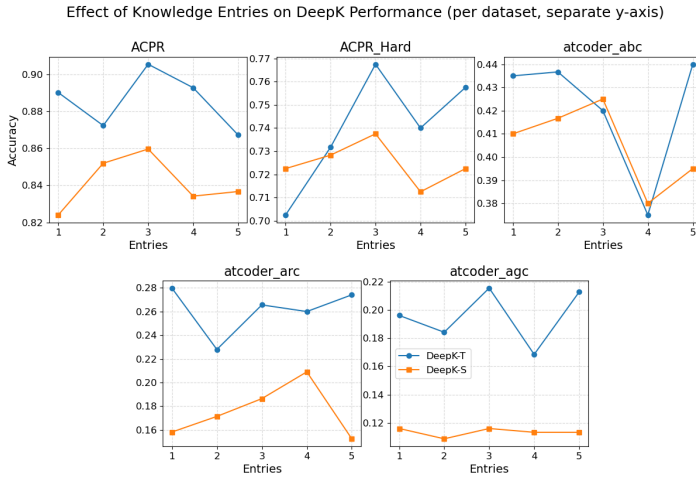


Fig. 5. The performance of DeepK-T and DeepK-S under DeepSeek-v3 with various number of retrieved knowledge entries.

Effect of Retrieval Size on Performance. For **DeepK-T**, performance fluctuates with the number of entries. While using five entries sometimes recovers performance close to the optimum, two and four entries consistently underperform across datasets. This suggests that excessive retrieval may introduce irrelevant or noisy information that dilutes the benefit of structured debugging knowledge. For **DeepK-S**, the trend is more consistent: performance steadily improves from one to three entries and declines when more entries are introduced. The three-entry configuration achieves the best overall result (average rank = 1.3), whereas four or five entries lead to notable degradation, particularly on out-of-distribution datasets such as Atcoder-ABC and Atcoder-ARC. This pattern indicates that DeepK-S is more sensitive to over-retrieval than DeepK-T.

Across datasets, the configuration with $k = 3$ consistently attains the best or near-best performance. For DeepK-T, the average rank is lowest at $k = 3$ (2.00), followed by $k = 5$ (2.40), whereas $k = 2$ and $k = 4$ are least effective (both 3.80). For DeepK-S, the preference for $k = 3$ is more pronounced (average rank = 1.30); larger k degrades performance ($k = 4$: 3.70; $k = 5$: 3.80).

Discussion. Overall, these results reveal a **U-shaped relationship** between the number of retrieved knowledge entries and repair performance: too few entries provide insufficient guidance, while too many entries introduce distracting noise. The optimal configuration of three entries provides the best balance between informativeness and precision, highlighting the importance of carefully tuning retrieval size when leveraging structured debugging knowledge.

6 Threats to Validity

While our experimental results demonstrate the effectiveness of DeepK, several threats to validity should be acknowledged.

Dataset Bias. Our evaluation leverages benchmark datasets including ACPR and Atcoder. Although these datasets contain plentiful buggy code items, they may not fully represent the complexity and diversity of real-world bugs. Nevertheless, our experiments demonstrate the effectiveness of integrating structured debugging knowledge into LLM-based APR systems. Additionally, we provide a systematic framework for extracting and structuring debugging knowledge from debugging code dataset, which can be readily reused to enhance other datasets and APR tasks.

Data Leakage. Some benchmark datasets may partially overlap with data seen during the pre-training of LLMs, an issue that is difficult to fully eliminate given the continual updates of large-scale corpora. Despite this potential leakage, our results show consistent gains across multiple models, datasets, and experimental settings, suggesting that the observed benefits are not solely attributable to data overlap. Furthermore, our datasets encompass a wide variety of coding problems and tasks, and we have deliberately curated newer data, such as AtCoder problems released after late 2023.

Function-Level Scope. Our current evaluation focuses on function-level repairs and does not directly address repository-level APR. Extending DeepK to repository-level APR would require new forms of knowledge representation to capture cross-file reasoning, as well as retrieval indices aligned with project-level structures. Recent systems such as SWE-Agent[48] organize multi-step history trajectories as demonstrations for large-scale repair; DeepK’s debugging knowledge could be reformulated into similar trajectory-style representations to augment such systems. Recent work SWE-Exp[7] further suggests the feasibility of integrating knowledge-centric enhancements into repository-level APR. We view this extension as a promising direction for future work.

Evaluation Metrics. We primarily report repair accuracy to measure program repair effectiveness. While these metrics provide a quantitative assessment, they may not fully capture partial correctness, semantic equivalence, or the human-perceived quality of patches. Incorporating complementary evaluation approaches, such as manual inspection or developer feedback, could provide a more nuanced understanding of repair quality.

7 Conclusion

This paper introduced DeepK, a RAG-based framework that systematically extracts, validates, and reuses structured debugging knowledge to enhance LLM-based APR. Evaluations demonstrate that DeepK consistently outperforms state-of-the-art systems in both in- and out-of-distribution scenarios, highlighting its robustness and scalability. Ablation studies confirm the critical contributions of multi-dimensional retrieval, the dual components of debugging knowledge (problem understanding and repair strategy), and the edit description module. Furthermore, varying the number of retrieved entries reveals a U-shaped effect: too few entries limit available context, while too many introduce noise, with three entries yielding the most reliable balance.

Overall, these findings demonstrate that structured debugging knowledge can significantly enhance LLM reasoning and repair. By shifting from single-instance reasoning to knowledge-guided repair, DeepK provides a principled and extensible pathway towards building more reliable and interpretable APR systems. Future work may extend this framework to repository-level repair, explore adaptive retrieval strategies for improved generalization, and integrate richer forms of software engineering knowledge.

8 Data Availability

The source code, data, and other artifacts have been made available at [our GitHub repository](#).

Acknowledgment

This paper is supported by the Key Research Program of Zhejiang Province (Grant No. 2026C01026), the National Natural Science Foundation of China (NSFC) under Grant No.(62572422, 62502439), the Zhejiang Provincial Natural Science Foundation of China under Grant No.LQN25F020009, the Yongjiang Talent Programme (2024A-160-G), Ant Group through CCF-Ant Research Fund.

References

- [1] AtCoder 2025. *AtCoder*. <https://atcoder.jp/>
- [2] Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, et al. 2021. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732* (2021).
- [3] Berkay Berabi, Jingxuan He, Veselin Raychev, and Martin Vechev. 2021. Tfix: Learning to fix coding errors with a text-to-text transformer. In *International Conference on Machine Learning*. PMLR, 780–791.
- [4] Sebastian Borgeaud, Arthur Mensch, Jordan Hoffmann, Trevor Cai, Eliza Rutherford, Katie Millican, George Bm Van Den Driessche, Jean-Baptiste Lespiau, Bogdan Damoc, Aidan Clark, et al. 2022. Improving language models by retrieving from trillions of tokens. In *International conference on machine learning*. PMLR, 2206–2240.
- [5] Islem Bouzenia, Premkumar Devanbu, and Michael Pradel. 2024. Repairagent: An autonomous, llm-based agent for program repair. *arXiv preprint arXiv:2403.17134* (2024).
- [6] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, Dave Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William Hebgen Guss, Alex Nichol, Alex Paino, Nikolas Tezak, Jie Tang, Igor Babuschkin, Suchir Balaji, Shantanu Jain, William Saunders, Christopher Hesse, Andrew N. Carr, Jan Leike, Josh Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever, and Wojciech Zaremba. 2021. Evaluating Large Language Models Trained on Code. *arXiv:2107.03374* [cs.LG] <https://arxiv.org/abs/2107.03374>
- [7] Silin Chen, Shaoxin Lin, Xiaodong Gu, Yuling Shi, Heng Lian, Longfei Yun, Dong Chen, Weiguo Sun, Lin Cao, and Qianxiang Wang. 2025. SWE-Exp: Experience-Driven Software Issue Resolution. *arXiv preprint arXiv:2507.23361* (07 2025). doi:10.48550/arXiv.2507.23361
- [8] Xinyun Chen, Maxwell Lin, Nathanael Schärli, and Denny Zhou. 2023. Teaching large language models to self-debug. *arXiv preprint arXiv:2304.05128* (2023).
- [9] Runxiang Cheng, Michele Tufano, Jürgen Cito, José Cambronero, Pat Rondon, Renyao Wei, Aaron Sun, and Satish Chandra. 2025. Agentic bug reproduction for effective automated program repair at google. *arXiv preprint arXiv:2502.01821* (2025).
- [10] Zhenlong Dai, Bingrui Chen, Zhuoluo Zhao, Xiu Tang, Sai Wu, Chang Yao, Zhipeng Gao, and Jingyuan Chen. 2025. Less is More: Adaptive Program Repair with Bug Localization and Preference Learning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 39. 128–136.
- [11] Jingwen Deng, Jihao Huang, Zhen Hao Wong, Hao Liang, Quanqing Xu, Bin Cui, and Wentao Zhang. 2025. Data-Centric Perspectives on Agentic Retrieval-Augmented Generation: A Survey. *Authorea Preprints* (2025).
- [12] Wenqi Fan, Yujian Ding, Liangbo Ning, Shijie Wang, Hengyun Li, Dawei Yin, Tat-Seng Chua, and Qing Li. 2024. A Survey on RAG Meeting LLMs: Towards Retrieval-Augmented Large Language Models. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining (Barcelona, Spain) (KDD '24)*. Association for Computing Machinery, New York, NY, USA, 6491–6501. doi:10.1145/3637528.3671470
- [13] Pengfei Gao, Zhao Tian, Xiangxin Meng, Xinchun Wang, Ruida Hu, Yuanan Xiao, Yizhou Liu, Zhao Zhang, Junjie Chen, Cuiyun Gao, et al. 2025. Trae Agent: An LLM-based Agent for Software Engineering with Test-time Scaling. *arXiv preprint arXiv:2507.23370* (2025).
- [14] Yunfan Gao, Yun Xiong, Xinyu Gao, Kangxiang Jia, Jinliu Pan, Yuxi Bi, Yixin Dai, Jiawei Sun, Haofen Wang, and Haofen Wang. 2023. Retrieval-augmented generation for large language models: A survey. *arXiv preprint arXiv:2312.10997* 2, 1 (2023).
- [15] Dong Huang, Jie M Zhang, Michael Luck, Qingwen Bu, Yuhao Qing, and Heming Cui. 2023. Agentcoder: Multi-agent-based code generation with iterative testing and optimisation. *arXiv preprint arXiv:2312.13010* (2023).
- [16] Jinhao Jiang, Kun Zhou, Zican Dong, Keming Ye, Wayne Xin Zhao, and Ji-Rong Wen. 2023. Structgpt: A general framework for large language model to reason over structured data. *arXiv preprint arXiv:2305.09645* (2023).
- [17] Nan Jiang and Yi Wu. 2024. RepairCAT: Applying Large Language Model to Fix Bugs in AI-Generated Programs. In *Proceedings of the 5th ACM/IEEE International Workshop on Automated Program Repair (Lisbon, Portugal) (APR '24)*. Association for Computing Machinery, New York, NY, USA, 58–60. doi:10.1145/3643788.3648020
- [18] Dongsun Kim, Jaechang Nam, Jaewoo Song, and Sunghun Kim. 2013. Automatic patch generation learned from human-written patches. In *Proceedings of the 2013 International Conference on Software Engineering (San Francisco, CA, USA) (ICSE '13)*. IEEE Press, 802a–811.
- [19] Jialong Kong, Mingfei Cheng, Xiaofei Xie, Shangqing Liu, Xiaoning Du, and Qi Guo. 2024. ContrastRepair: Enhancing Conversation-Based Automated Program Repair via Contrastive Test Case Pairs. *arXiv:2403.01971* [cs.SE] <https://arxiv.org/abs/2403.01971>

- [20] ShareAI Lab. 2024. Kode: An Open-Source Agentic Coding Framework. <https://github.com/shareAI-lab/Kode>. Accessed: 2025-09-12.
- [21] Xuan Bach D. Le, David Lo, and Claire Le Goues. 2016. History Driven Program Repair. In *2016 IEEE 23rd International Conference on Software Analysis, Evolution, and Reengineering (SANER)*, Vol. 1. 213–224. doi:10.1109/SANER.2016.76
- [22] Claire Le Goues, ThanhVu Nguyen, Stephanie Forrest, and Westley Weimer. 2012. GenProg: A Generic Method for Automatic Software Repair. *IEEE Transactions on Software Engineering* 38, 1 (2012), 54–72. doi:10.1109/TSE.2011.104
- [23] Claire Le Goues, Michael Pradel, and Abhik Roychoudhury. 2019. Automated program repair. *Commun. ACM* 62, 12 (Nov. 2019), 56–65. doi:10.1145/3318162
- [24] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. 2020. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Proceedings of the 34th International Conference on Neural Information Processing Systems (Vancouver, BC, Canada) (NIPS '20)*. Curran Associates Inc., Red Hook, NY, USA, Article 793, 16 pages.
- [25] Kui Liu, Anil Koyuncu, Dongsun Kim, and Tegawendé F Bissyandé. 2019. Avatar: Fixing semantic bugs with fix patterns of static analysis violations. In *2019 IEEE 26th International Conference on Software Analysis, Evolution and Reengineering (SANER)*. IEEE, 1–12.
- [26] Fan Long and Martin Rinard. 2015. Staged program repair with condition synthesis. In *Proceedings of the 2015 10th Joint Meeting on Foundations of Software Engineering (Bergamo, Italy) (ESEC/FSE 2015)*. Association for Computing Machinery, New York, NY, USA, 166–178. doi:10.1145/2786805.2786811
- [27] Fan Long and Martin Rinard. 2015. Staged program repair with condition synthesis. In *Proceedings of the 2015 10th Joint Meeting on Foundations of Software Engineering (Bergamo, Italy) (ESEC/FSE 2015)*. Association for Computing Machinery, New York, NY, USA, 166–178. doi:10.1145/2786805.2786811
- [28] Aditi Mavalankar, Hassan Mansoor, Zita Marinho, Masha Samsikova, and Tom Schaul. 2025. AuPair: Golden Example Pairs for Code Repair. *arXiv preprint arXiv:2502.18487* (2025).
- [29] Martin Monperrus. 2018. Automatic Software Repair: A Bibliography. *ACM Comput. Surv.* 51, 1, Article 17 (Jan. 2018), 24 pages. doi:10.1145/3105906
- [30] Maxwell Nye, Anders Johan Andreassen, Guy Gur-Ari, Henryk Michalewski, Jacob Austin, David Bieber, David Dohan, Aitor Lewkowycz, Maarten Bosma, David Luan, et al. 2021. Show your work: Scratchpads for intermediate computation with language models. (2021).
- [31] Theo X Olausson, Jeevana Priya Inala, Chenglong Wang, Jianfeng Gao, and Armando Solar-Lezama. 2024. Is Self-Repair a Silver Bullet for Code Generation?. In *ICLR*.
- [32] Ruchir Puri, David S Kung, Geert Janssen, Wei Zhang, Giacomo Domeniconi, Vladimir Zolotov, Julian Dolby, Jie Chen, Mihir Choudhury, Lindsey Decker, et al. 2021. Codenet: A large-scale ai for code dataset for learning a diversity of coding tasks. *arXiv preprint arXiv:2105.12655* (2021).
- [33] Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J Liu. 2020. Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of machine learning research* 21, 140 (2020), 1–67.
- [34] Kurt Shuster, Spencer Poff, Moya Chen, Douwe Kiela, and Jason Weston. 2021. Retrieval Augmentation Reduces Hallucination in Conversation. In *Findings of the Association for Computational Linguistics: EMNLP 2021*, Marie-Francine Moens, Xuanjing Huang, Lucia Specia, and Scott Wen-tau Yih (Eds.). Association for Computational Linguistics, Punta Cana, Dominican Republic, 3784–3803. doi:10.18653/v1/2021.findings-emnlp.320
- [35] André Silva, Sen Fang, and Martin Monperrus. 2025. Repairllama: Efficient representations and fine-tuned adapters for program repair. *IEEE Transactions on Software Engineering* (2025).
- [36] Edward K. Smith, Earl T. Barr, Claire Le Goues, and Yuriy Brun. 2015. Is the cure worse than the disease? overfitting in automated program repair. In *Proceedings of the 2015 10th Joint Meeting on Foundations of Software Engineering (Bergamo, Italy) (ESEC/FSE 2015)*. Association for Computing Machinery, New York, NY, USA, 532–543. doi:10.1145/2786805.2786825
- [37] Yaodong Su, Yixiang Fang, Yingli Zhou, Quanqing Xu, and Chuanhui Yang. 2025. Clue-RAG: Towards Accurate and Cost-Efficient Graph-based RAG via Multi-Partite Graph and Query-Driven Iterative Retrieval. *arXiv preprint arXiv:2507.08445* (2025).
- [38] Wei Tao, Yucheng Zhou, Yanlin Wang, Wenqiang Zhang, Hongyu Zhang, and Yu Cheng. 2024. Magis: Llm-based multi-agent framework for github issue resolution. *Advances in Neural Information Processing Systems* 37 (2024), 51963–51993.
- [39] Xingyao Wang, Boxuan Li, Yufan Song, Frank F Xu, Xiangru Tang, Mingchen Zhuge, Jiayi Pan, Yueqi Song, Bowen Li, Jaskirat Singh, et al. 2024. Openhands: An open platform for ai software developers as generalist agents. *arXiv preprint arXiv:2407.16741* (2024).
- [40] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. 2022. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing*

- systems 35 (2022), 24824–24837.
- [41] Robert F Woolson. 2007. Wilcoxon signed-rank test. *Wiley encyclopedia of clinical trials* (2007), 1–3.
 - [42] Chunqiu Steven Xia, Yinlin Deng, Soren Dunn, and Lingming Zhang. 2024. Agentless: Demystifying llm-based software engineering agents. *arXiv preprint arXiv:2407.01489* (2024).
 - [43] Chunqiu Steven Xia and Lingming Zhang. 2022. Less training, more repairing please: revisiting automated program repair via zero-shot learning. In *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 959–971.
 - [44] Chunqiu Steven Xia and Lingming Zhang. 2024. Automated Program Repair via Conversation: Fixing 162 out of 337 Bugs for \$0.42 Each using ChatGPT. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis* (Vienna, Austria) (ISSTA 2024). Association for Computing Machinery, New York, NY, USA, 819–831. doi:10.1145/3650212.3680323
 - [45] Yahnoosh, HeidiSteen. 2025. Relevance scoring in hybrid search using Reciprocal Rank Fusion (RRF). <https://learn.microsoft.com/en-us/azure/search/hybrid-search-ranking>. Accessed: 2025-04-24.
 - [46] Boyang Yang, Haoye Tian, Weiguo Pian, Haoran Yu, Haitao Wang, Jacques Klein, Tegawendé F. Bissyandé, and Shunfu Jin. 2024. CREF: An LLM-Based Conversational Software Repair Framework for Programming Tutors. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis* (Vienna, Austria) (ISSTA 2024). Association for Computing Machinery, New York, NY, USA, 882–894. doi:10.1145/3650212.3680328
 - [47] Boyang Yang, Haoye Tian, Jiadong Ren, Shunfu Jin, Yang Liu, Feng Liu, and Bach Le. 2025. Enhancing Repository-Level Software Repair via Repository-Aware Knowledge Graphs. arXiv:2503.21710 [cs.SE] <https://arxiv.org/abs/2503.21710>
 - [48] John Yang, Carlos E Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. 2024. Swe-agent: Agent-computer interfaces enable automated software engineering. *Advances in Neural Information Processing Systems* 37 (2024), 50528–50652.
 - [49] Xin Yin, Chao Ni, Shaohua Wang, Zhenhao Li, Limin Zeng, and Xiaohu Yang. 2024. Thinkrepair: Self-directed automated program repair. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*. 1274–1286.
 - [50] Wei Yuan, Quanjun Zhang, Tiek He, Chunrong Fang, Nguyen Quoc Viet Hung, Xiaodong Hao, and Hongzhi Yin. 2022. CIRCLE: continual repair across programming languages. In *Proceedings of the 31st ACM SIGSOFT International Symposium on Software Testing and Analysis* (Virtual, South Korea) (ISSTA 2022). Association for Computing Machinery, New York, NY, USA, 678–690. doi:10.1145/3533767.3534219
 - [51] Qiming Zeng, Xiao Yan, Hao Luo, Yuhao Lin, Yuxiang Wang, Fangcheng Fu, Bo Du, Quanqing Xu, and Jiawei Jiang. 2025. How Significant Are the Real Performance Gains? An Unbiased Evaluation Framework for GraphRAG. *arXiv preprint arXiv:2506.06331* (2025).
 - [52] K. Zhang and D. Shasha. 1989. Simple fast algorithms for the editing distance between trees and related problems. *SIAM J. Comput.* 18, 6 (Dec. 1989), 1245–1262. doi:10.1137/0218082
 - [53] Yuwei Zhang, Zhi Jin, Ying Xing, Ge Li, Fang Liu, Jiaxin Zhu, Wensheng Dou, and Jun Wei. 2025. PATCH: Empowering Large Language Model with Programmer-Intent Guidance and Collaborative-Behavior Simulation for Automatic Bug Fixing. *ACM Trans. Softw. Eng. Methodol.* (Feb. 2025). doi:10.1145/3718739 Just Accepted.
 - [54] Yuntong Zhang, Haifeng Ruan, Zhiyu Fan, and Abhik Roychoudhury. 2024. Autocoderover: Autonomous program improvement. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*. 1592–1604.
 - [55] Juang Zhao, Donghao Yang, Li Zhang, Xiaoli Lian, Zitian Yang, and Fang Liu. 2024. Enhancing Automated Program Repair with Solution Design. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering* (Sacramento, CA, USA) (ASE '24). Association for Computing Machinery, New York, NY, USA, 1706–1718. doi:10.1145/3691620.3695537
 - [56] Li Zhong, Zilong Wang, and Jingbo Shang. 2024. Debug like a human: A large language model debugger via verifying runtime execution step-by-step. *arXiv preprint arXiv:2402.16906* (2024).
 - [57] Xin Zhou, Kisub Kim, Bowen Xu, Donggyun Han, and David Lo. 2024. Out of Sight, Out of Mind: Better Automatic Vulnerability Repair by Broadening Input Ranges and Sources. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering* (Lisbon, Portugal) (ICSE '24). Association for Computing Machinery, New York, NY, USA, Article 88, 13 pages. doi:10.1145/3597503.3639222

Received 2025-09-12; accepted 2025-12-22